

Spring Data JPA with Spring Boot, Hibernate | Cognizant DN 5.0

Sanskar Kumawat

HANDSON 1 – Spring Data JPA Quick Example

Aim

To develop a Spring Boot application using **Spring Data JPA** and **Hibernate** that connects to a **MySQL** database, retrieves records from the **Country** table, and demonstrates Object Relational Mapping (ORM).

Objectives

1. Understand the need for Object Relational Mapping (ORM).
2. Learn the advantages of Spring Data JPA.
3. Understand Hibernate architecture.
4. Create a Spring Boot project using Spring Initializr.
5. Configure MySQL database connectivity.
6. Create a Country entity.
7. Create Repository and Service classes.
8. Retrieve data from the database using JpaRepository.
9. Execute the application successfully.

What is ORM?

ORM (Object Relational Mapping) is a technique used to map Java objects to relational database tables.

Instead of writing SQL queries manually, developers work with Java objects and the ORM framework automatically converts object operations into SQL queries.

Example:

Java Object

Country

↓

Database Table

country

Advantages

- Reduces SQL coding
- Faster development

- Database independent
- Easy maintenance
- Supports transactions
- Reduces boilerplate code

What is JPA?

JPA stands for **Java Persistence API**.

It is a specification that defines how Java objects should be stored and retrieved from relational databases.

Important point:

JPA is only a specification.

It does not contain any implementation.

Examples of JPA implementations

- Hibernate
- EclipseLink
- OpenJPA

What is Hibernate?

Hibernate is an ORM framework.

It is the most popular implementation of JPA.

Responsibilities

- Converts Java objects into SQL
- Generates SQL queries
- Manages sessions
- Handles transactions
- Maps entities to database tables

What is Spring Data JPA?

Spring Data JPA is built on top of JPA.

It removes boilerplate code by providing ready-made repository methods.

Instead of writing

```
SELECT * FROM country
```

You simply write

```
countryRepository.findAll();
```

Advantages

- Less code
- Automatic CRUD operations
- Transaction management
- Easy integration with Spring Boot
- Faster development

Difference between JPA, Hibernate and Spring Data JPA

JPA	Hibernate	Spring Data JPA
Specification	ORM Framework	Spring Module
No implementation	Implements JPA	Uses Hibernate internally
Defines standards	Generates SQL	Provides Repository methods
No CRUD implementation	CRUD possible	CRUD already available

Handson 1 – Steps Performed

Step 1: Created a Spring Boot Maven Project

- Created a new Maven project named **orm-learn**.
- Set the package name as:

```
com.cognizant.ormlearn
```

Step 2: Added Required Dependencies

Modified pom.xml and added:

- Spring Boot Starter Data JPA
- MySQL Connector/J
- Spring Boot Starter Test

Also configured:

- Java 17
- Spring Boot parent

- Maven plugins

Step 3: Configured MySQL Database

Created application.properties and configured:

- Database URL
- Username
- Password
- Hibernate DDL Auto
- SQL Logging

Example:

```
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
```

```
spring.datasource.username=root
```

```
spring.datasource.password=*****
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

Step 4: Created Database

Opened MySQL.

Created database:

```
CREATE DATABASE ormlearn;
```

Selected database:

```
USE ormlearn;
```

```
]
```

Step 5: Created Country Table

Created the table:

```
CREATE TABLE country(  
    code VARCHAR(2) PRIMARY KEY,  
    name VARCHAR(50)  
);
```

Step 6: Inserted Sample Records

Inserted two records:

```
INSERT INTO country VALUES ('IN','India');
```

```
INSERT INTO country VALUES ('US','United States of America');
```

Step 7: Created Model Package

Created package:

```
com.cognizant.ormlearn.model
```

Step 8: Created Country Entity

Created

```
Country.java
```

Mapped the class using

- @Entity
- @Table
- @Id
- @Column

Fields:

```
private String code;
```

```
private String name;
```

Generated

- Constructors
- Getters
- Setters
- toString()

Step 9: Created Repository Package

Created package

```
com.cognizant.ormlearn.repository
```

Step 10: Created CountryRepository

Created interface

CountryRepository

Extended

JpaRepository<Country,String>

Added

@Repository

Step 11: Created Service Package

Created package

com.cognizant.ormlearn.service

Step 12: Created CountryService

Created class

CountryService

Added

@Service

Injected

CountryRepository

using

@Autowired

Created method

getAllCountries()

Annotated with

@Transactional

Used

countryRepository.findAll();

to fetch records.

Step 13: Modified Main Application

Opened

OrmLearnApplication.java

Added

ApplicationContext

Obtained

CountryService

bean

Created

testGetAllCountries()

method

Called

countryService.getAllCountries();

Displayed all country records.

Step 14: Built the Project

Executed

mvn clean

Verified there were no compilation errors.

Step 15: Ran the Application

Executed

mvn spring-boot:run

Step 16: Verified Output

Observed:

- Spring Boot started successfully.
- Hibernate connected to MySQL.
- SQL query executed:

select

c1_0.code,

c1_0.name

from country c1_0;

Output displayed:

Country{code='IN', name='India'}

Country{code='US', name='United States of America'}

Step 17: Verified Database

Opened MySQL and executed:

USE ormlearn;

SHOW TABLES;

SELECT * FROM country;